CARAGA STATE UNIVERSITY
COLLEGE OF COMPUTING AND INFORMATION SCIENCES
CSU-Main Campus, Ampayon, Butuan City, Philippines
Competence Service Uprightness
YOUR BEST OPTION TO SUCCESS

ITE 18 – Application Development and Emerging Technologies

MIDTERM PROJECT DOCUMENTATION

BACKEND TRACK

Andrea Marx C. Cago

Rowella J. Consigna

Specialization: Backend

ITE 18 - AQGW1

Glaicel Araneta

October 16, 2026

Semester, A.Y. 2026–2027

I. Project Overview

Project description

The Mining Equipment Management System (MEMS) is a proposed backend system for organizing mining equipment, site assignments, maintenance schedules and records, and inspection results. Mining operations often keep equipment information in separate files or manual records, making maintenance history and equipment accountability difficult to track. MEMS will centralize these records in PostgreSQL and provide secure RESTful API access through NestJS, JWT authentication, and role-based permissions.

Objectives

General Objective

The general objective is to design a secure backend for managing mining equipment information.

Specific Objectives

1. Manage equipment, categories, and sites.
2. Record assignments and responsible personnel.
3. Schedule and document maintenance.
4. Record inspection findings
5. Provide role-controlled CRUD endpoints and basic reports.
6. Prepare validation, testing, and deployment plans for the Final.

Scope & limitations

Scope

- Equipment inventory, categories, and assignments
- Mining sites
- Maintenance records, schedules, and inspections
- User accounts, JWT, audit logs, basic reports
- Backend API designed and planned testing

Limitation

- Live GPS tracking and equipment sensor integration
- Mining production, payroll, and full accounting
- Automatic spare-parts replenishment and AI failure prediction
- Dedicated mobile application

II. System Design & Development

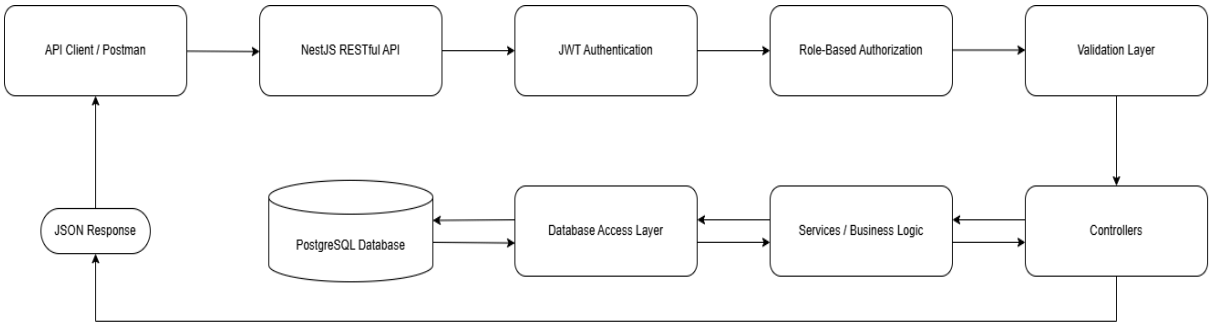
A. System Architecture

Backend architecture

MEMS will use a modular NestJS RESTful API running on Node.js with TypeScript and PostgreSQL. Requests pass through JWT authentication, role checks and validation before

controllers call services and the database access layer. Controllers handle HTTP requests; services apply business rules; the database enforces keys and constraints.

System architecture diagram



Chosen framework

Component	Technology	Purpose
Runtime environment	Node.js	Executes the compiled backend application
Backend framework	NestJS	Organizes the backend into modules, controllers, services, and providers
Programming language	TypeScript	Adds static typing to the backend application logic
API architecture	RESTful API	Provides HTTP endpoints for creating, retrieving, updating, and deleting records
Database management system	PostgreSQL	Stores relational data with constraints and relationships

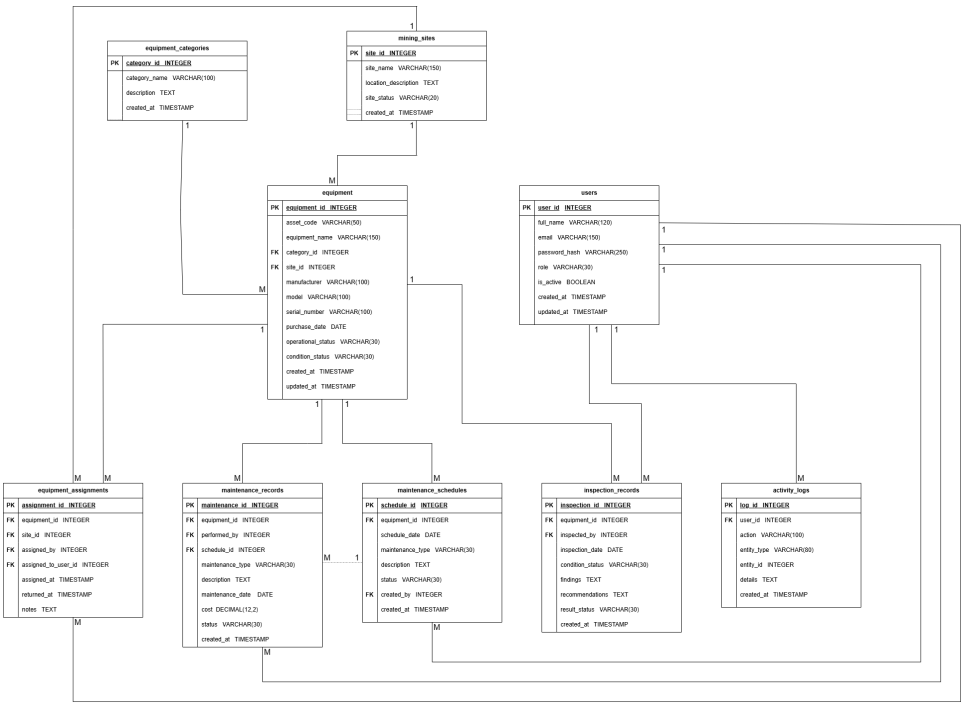
Authentication	JSON Web Token (JWT)	Authenticates users accessing protected endpoints
Authorization	Role-Based Access Control	Restricts system operations according to user roles
API testing and documentation	Postman	Tests endpoints and documents requests, responses, and authentication
Version control	Git and GitHub	Tracks code changes and supports collaboration between group members
Hosting environment	Local server	Runs the backend and database locally for development and demonstration

Planned Folder Structure

```
mining-equipment-management-system/
├── src/
│   ├── auth/
│   │   ├── auth.controller.ts
│   │   ├── auth.service.ts
│   │   └── auth.module.ts
│   ├── users/
│   │   ├── users.controller.ts
│   │   ├── users.service.ts
│   │   └── users.module.ts
│   ├── equipment/
│   │   ├── equipment.controller.ts
│   │   ├── equipment.service.ts
│   │   └── equipment.module.ts
│   ├── equipment-categories/
│   ├── mining-sites/
│   ├── equipment-assignments/
```

```
│   ├── maintenance-records/
│   ├── maintenance-schedules/
│   ├── inspection-records/
│   ├── activity-logs/
│   ├── database/
│   ├── common/
│   │   ├── guards/
│   │   ├── decorators/
│   │   ├── filters/
│   │   └── validation/
│   ├── app.module.ts
│   └── main.ts
├── docs/
├── test/
├── postman/
├── .env.example
├── .gitignore
├── package.json
└── README.md
```

B. Database Design



The database has nine related tables. Equipment belongs to one category and one current site; each equipment item can have multiple assignment, maintenance, schedule and inspection records. Users are linked to assignments, maintenance, schedules, inspections and logs. A maintenance record can optionally refer to a planned schedule.

Explanation of table relationships and dependencies

Relationship	Cardinality	Foreign keys
equipment_categories → equipment	One-to-many	equipment.category_id
mining_sites → equipment	One-to-many	equipment.site_id
equipment → equipment_assignments	One-to-many	equipment_assignments.equipment_id
mining_sites → equipment_assignments	One-to-many	equipment_assignments.site_id
users → equipment_assignments	One-to-many	via assigned_by and assigned_to_user_id
equipment → maintenance_records	One-to-many	via equipment_id
users → maintenance_records	One-to-many	via performed_by

equipment → maintenance_schedules	One-to-many	via equipment_id
users → maintenance_schedules	One-to-many	via created_by
maintenance_schedules → maintenance_records	One-to-many (Optional)	optional via maintenance_records.schedule_id
equipment → inspection_records	One-to-many	via equipment_id
users → inspection_records	One-to-many	via inspected_by
users → activity_logs	One-to-many	via activity_logs.user_id

The equipment_assignments table will contain two distinct user foreign keys:

- assigned_by references the user who created the assignment.
- assigned_to_user_id references the user responsible for the assigned equipment.

This distinction is important because the person assigning the equipment may be different from the person receiving it.

The activity_logs table will use user_id to reference the user who performed an action. Its entity_type and entity_id fields will identify the affected record. Because the affected record can belong to different tables, entity_id will be treated as a generic reference rather than a conventional foreign key to one specific table.

Overview of the nine database tables

No.	Table name	Purpose	Primary key
1	users	Stores system user accounts, password hashes, roles, and account status.	user_id
2	equipment_categories	Stores equipment classifications, such as excavators, dump trucks, and bulldozers.	category_id
3	mining_sites	Stores the names and locations of mining sites.	site_id
4	equipment	Stores equipment identification, specifications, location, condition, and operational status.	equipment_id
5	equipment_assignments	Records equipment assignments, responsible users, destinations, and assignment history.	assignment_id
6	maintenance_records	Stores details of maintenance activities performed on equipment.	maintenance_id
7	maintenance_schedules	Stores planned equipment maintenance and scheduled service dates.	schedule_id
8	inspection_records	Stores inspection dates, findings, equipment condition, and recommendations.	inspection_id
9	activity_logs	Records important user actions for accountability and auditing.	log_id

Normalization applied

The design targets **Third Normal Form (3NF)**: **1NF** uses atomic fields and one record per row; **2NF** avoids partial dependencies (all proposed primary keys are single-column); **3NF** separates category and site names into their own tables rather than repeating them in equipment. Maintenance, schedules and inspections are stored separately to preserve history. The application must keep current equipment state consistent with assignment history using transactions.

Selected DBMS: PostgreSQL

PostgreSQL is selected for structured, related records. Each table uses a single-column INTEGER identity primary key; foreign keys maintain references. VARCHAR, TEXT, DATE, TIMESTAMP, BOOLEAN and DECIMAL(12,2) support the planned data. UNIQUE, NOT NULL, CHECK and DEFAULT constraints protect integrity.

C. API Development

API Routes, Endpoints, and HTTP Methods

The following endpoints are planned for the Final; they are not implemented at Midterm. All protected endpoints require a valid JWT. "All" means all authenticated roles. ID routes use :id parameters.

Method	Endpoint	Description	Access
POST	/api/auth/login	Authenticates a user and returns an access token.	Public
GET	/api/users	Retrieves all user accounts.	Admin
GET	/api/users/:id	Retrieves a specific user account.	Admin
POST	/api/users	Creates a new user account.	Admin
PATCH	/api/users/:id	Updates user information or role.	Admin

DELETE	/api/users/:id	Deactivates a user account while preserving historical records.	Admin
GET	/api/equipment-categories	Retrieves all equipment categories.	All authenticated roles
GET	/api/equipment-categories/:id	Retrieves a specific equipment category.	All authenticated roles
POST	/api/equipment-categories	Creates a new equipment category.	Admin
PATCH	/api/equipment-categories/:id	Updates equipment category information.	Admin
DELETE	/api/equipment-categories/:id	Deletes a category only if no existing equipment references it.	Admin
GET	/api/mining-sites	Retrieves all mining sites.	All authenticated roles
GET	/api/mining-sites/:id	Retrieves a specific mining site.	All authenticated roles
POST	/api/mining-sites	Registers a new mining site.	Admin

PATCH	/api/mining-sites/:id	Updates mining site information or status.	Admin
DELETE	/api/mining-sites/:id	Deletes a mining site if no existing records depend on it.	Admin
GET	/api/equipment	Retrieves all equipment records.	All authenticated roles
GET	/api/equipment/:id	Retrieves a specific equipment record.	All authenticated roles
POST	/api/equipment	Registers new mining equipment.	Admin, Equipment Manager
PATCH	/api/equipment/:id	Updates equipment information, condition, or operational status.	Admin, Equipment Manager
DELETE	/api/equipment/:id	Deletes equipment only when removal is permitted and historical dependencies are preserved.	Admin
GET	/api/equipment-assignments	Retrieves equipment assignment records.	All authenticated roles

GET	/api/equipment-assignments/:id	Retrieves a specific equipment assignment.	All authenticated roles
POST	/api/equipment-assignments	Assigns equipment to a mining site and responsible users.	Admin, Equipment Manager
PATCH	/api/equipment-assignments/:id	Updates assignment information or records an equipment return.	Admin, Equipment Manager
DELETE	/api/equipment-assignments/:id	Deletes an assignment record only when authorized and when deletion will not violate database constraints or required record-retention rules.	Admin
GET	/api/maintenance-records	Retrieves equipment maintenance history.	All authenticated roles
GET	/api/maintenance-records/:id	Retrieves a specific maintenance record.	All authenticated roles
POST	/api/maintenance-records	Records maintenance performed on equipment.	Admin, Maintenance Staff
PATCH	/api/maintenance-records/:id	Updates maintenance details, cost, or status when authorized.	Admin, Maintenance Staff

DELETE	/api/maintenance-records/:id	Deletes an incorrect maintenance record only when authorized and when deletion will not violate database constraints or required record-retention rules.	Admin
GET	/api/maintenance-schedules	Retrieves planned maintenance schedules.	All authenticated roles
GET	/api/maintenance-schedules/:id	Retrieves a specific maintenance schedule.	All authenticated roles
POST	/api/maintenance-schedules	Creates a maintenance schedule for equipment.	Admin, Equipment Manager
PATCH	/api/maintenance-schedules/:id	Updates a schedule's date, details, or status.	Admin, Equipment Manager
DELETE	/api/maintenance-schedules/:id	Deletes a maintenance schedule only when no maintenance records reference it and deletion does not violate database constraints or required record-retention rules.	Admin, Equipment Manager
GET	/api/inspection-records	Retrieves equipment inspection records.	All authenticated roles

GET	/api/inspection-records/:id	Retrieves a specific inspection record.	All authenticated roles
POST	/api/inspection-records	Records inspection findings and recommendations.	Admin, Maintenance Staff
PATCH	/api/inspection-records/:id	Updates inspection information when authorized.	Admin, Maintenance Staff
DELETE	/api/inspection-records/:id	Deletes an incorrect inspection record only when authorized and when deletion does not violate database constraints or required record-retention rules.	Admin
GET	/api/activity-logs	Retrieves activity logs for auditing and monitoring user actions.	Admin
GET	/api/activity-logs/:id	Retrieves a specific activity log.	Admin

Audit logs are system-generated and should not have public create/update/delete routes. User DELETE deactivates accounts; equipment, assignments, and historical records must not be deleted when retention or foreign-key requirements would be violated. Basic reporting may be exposed through authenticated filtered GET requests.

Authentication and Authorization Strategy

POST /api/auth/login checks credentials against securely stored password hashes and issues a signed JWT. A JWT guard validates tokens on protected endpoints; an RBAC guard checks Admin, Equipment Manager and Maintenance Staff permissions. Missing/invalid tokens return 401; valid tokens without permission return 403. Admin manages users, categories and sites; Equipment Manager manages equipment and assignments; Maintenance Staff manages maintenance and inspections.

Sample API Request and Response

The following examples illustrate the planned request and response formats for the final implementation.

Example 1: Successful Equipment Registration

HTTP Method: POST

Endpoint: /api/equipment

Headers:

Authorization: Bearer <access_token>
Content-Type: application/json

Request Body:

```
{
        "asset_code": "EQ-001",
        "equipment_name": "Excavator 01",
        "category_id": 1,
        "site_id": 1,
        "manufacturer": "Caterpillar",
        "model": "320D",
        "serial_number": "CAT320D001",
        "purchase_date": "2026-01-15",
        "operational_status": "Available",
        "condition_status": "Good"
}
```

Expected Response:

HTTP Status: 201 Created

```
{
        "message": "Equipment registered successfully",
        "data": {
                "equipment_id": 1,
```

```
        "asset_code": "EQ-001",
        "equipment_name": "Excavator 01",
        "category_id": 1,
        "site_id": 1,
        "manufacturer": "Caterpillar",
        "model": "320D",
        "serial_number": "CAT320D001",
        "purchase_date": "2026-01-15",
        "operational_status": "Available",
        "condition_status": "Good"
    }
}
```


This response indicates that the equipment information has passed validation and has been successfully saved to the database. The system returns the newly created record and its assigned equipment ID.

Example 2: Error Response for Invalid Equipment Data

HTTP Method: POST

Endpoint: /api/equipment

Request Body:


```
{
    "asset_code": " " ,
    "equipment_name": "Excavator 01",
    "category_id": 999,
    "site_id": 1
}
```


Expected Response:

HTTP Status: 400 Bad Request


```
{
    "statusCode": 400,
    "message": [
        "asset_code should not be empty",
        "category_id must reference an existing category"
    ],
    "error": "Bad Request"
}
```

This error response indicates that the request contains invalid equipment information. The asset code is empty, and the specified category ID does not correspond to an existing equipment category. The backend will reject the request and prevent the invalid record from being saved.

The exact validation messages will depend on the validation rules implemented in NestJS. The backend will validate required fields and verify foreign-key references before saving records to the database.

D. Security, Deployment Plan & Version Control

Planned security practices

Validation: NestJS DTO validation will check required fields, types, valid IDs, allowed status values, nonnegative maintenance costs, and assignment dates. Use parameterized database access through the ORM; return consistent 400, 401, 403, 404, 409 and 500 errors without exposing stack traces or secrets.

Security: hash passwords, verify JWT signatures and expiration, apply least-privilege role guards, store secrets in environment variables excluded from Git, use HTTPS for network deployments, encrypt sensitive backups where appropriate, and restrict direct database access. Preserve maintenance/inspection history and protect audit logs against unauthorized edits.

Planned hosting

Deployment: develop and demonstrate on a local server with PostgreSQL and NestJS. Prepare .env.example, installation steps, and configuration for possible online hosting during the Final. Use Postman for API documentation/tests and GitHub for version control; keep main as stable branch and use feature branches and pull-request review. Back up database exports and repository content; test restoration.

Version control and backup

The Mining Equipment Management System (MEMS) will use Git and GitHub for version control and collaboration. The repository will store the project's documentation, database design, architecture diagrams, API specifications, and future backend source code. The team will use the main branch for stable project files and separate development branches for future implementation. GitHub will help both members track changes, organize their contributions, and maintain previous versions of the project.

GitHub repository URL:
https://github.com/andreamarxcago-ai/Mining-Equipment-Management-System

E. Testing Plan

The Mining Equipment Management System (MEMS) will use Postman to test the RESTful API endpoints during the final implementation phase.

Planned API Testing

Area	Planned test	Expected result
CRUD	Create, list, view, edit and allowed delete/deactivate for each entity	Correct JSON and database changes
Authentication	Valid, invalid, missing and expired JWT	Authorized access or 401
Authorization	Try restricted endpoint using another role	403 Forbidden
Validation	Missing fields, wrong types, duplicate asset code, invalid dates/cost	400 or 409; no invalid data saved
Relationships	Unknown category/equipment/site IDs; active assignment consistency	Rejected invalid references; consistent records
Error handling	Missing ID, database failure	404 or safe 500 response

III. Ethical and Professional Considerations

Ethical coding and data protection

MEMS will collect only user details needed for accounts and accountability, including name, email, role and activity history. Access will be restricted by role; passwords will be stored only as hashes. Data handling will consider the Philippine Data Privacy Act of 2012 (Republic Act No. 10173), including purpose limitation, appropriate security and access controls, and responsible retention. Real equipment inspections and safety decisions remain subject to qualified personnel and applicable safety procedures; the system does not certify equipment safety. Third-party libraries and tools will be used in accordance with their licenses.

IV. Midterm Progress Report

Design Item	Status	Notes
Project Overview and Scope	Done	The project overview, objectives, scope, and limitations of the Mining Equipment Management System (MEMS) are documented.
Entity-Relationship Diagram (ERD)	Done	The ERD identifies the system entities, primary key, foreign keys, and

		relationships.
Data Dictionary and Normalization	Done	The data dictionary describes the database fields, and the tables are designed using Third Normal Form (3NF).
System Architecture Explanation	Done	The system architecture and the roles of its major components are explained.
System Architecture Diagram	Done	The architecture diagram illustrates the planned structure and interactions of the system components.
API Development Plan	Done	The planned API endpoints, HTTP methods, access permissions, and sample request and response formats are documented.
Security and Deployment Plan	Done	The planned authentication, role-based access control, input validation, secure configuration, and hosting approach are described.
Version Control and Backup Plan	Done	The planned GitHub repository workflow, commit practices, and backup and recovery procedures are documented.
API Testing Plan	Done	The planned tests cover API functionality, authentication, authorization, input validation, error handling, and database integrity.
Ethical and Professional Considerations	Done	Data privacy, confidentiality, responsible data handling, and applicable privacy requirements are discussed.

Completed design work includes the project scope, nine-table ERD/schema, data dictionary, modular NestJS architecture, planned routes, JWT/RBAC plan, security plan and testing plan. These are design deliverables, not evidence of a working backend. Challenges included separating maintenance schedules from actual work, preserving assignment history, and identifying role-specific access. The design addresses these using separate tables, optional schedule links and RBAC.

Team Contributions

Member	Documented contribution
Andrea Marx C. Cago	Project overview, system architecture and diagram, technologies, folder structure, ERD, normalization and data dictionary.
Rowella J. Consigna	API routes and examples, security, deployment, version control, testing and ethical considerations.
Both members	Reviewed integration and design decisions.

V. Remaining Work and Plan for Final

Task	Responsible member	Target
Implement NestJS modules and PostgreSQL schema	Both members	Before Final submission
JWT, RBAC, DTO validation and safe errors	Rowella J. Consigna	Before Final submission
Database integration and CRUD routes	Andrea Marx C. Cago / both	Before Final submission
Postman testing, bug fixes and documentation	Both members	Before Final submission
Local demonstration, GitHub updates and backups	Both members	Before Final submission

Appendix

Appendix A: Github repository

GitHub repository URL:
https://github.com/andreamarxcago-ai/Mining-Equipment-Management-System